Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Факультет электронно-информационных систем

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Сравнение классических методов классификации

Специальность АС-68

Выполнил:

Студент 3-го курса

Группы АС-68

Богданов В. С.

Проверил:

А. А. Крощенко

Доцент кафедры ИИТ,

«___»_____ 2026 г.

Цель работы: на практике сравнить работу нескольких алгоритмов классификации: метода k-ближайших соседей (k-NN), деревьев решений и метода опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Общее задание:

- загрузить датасет по варианту;
- разделить данные на обучающую и тестовую выборки;
- обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM;
- для модели k-NN исследовать, как меняется качество при разном количестве соседей (k);
- оценить точность каждой модели на тестовой выборке;
- сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 1

Классификация видов ириса (Iris)

- Iris
- Определить вид ириса (setosa, versicolor, virginica) по измерениям его цветка
- **Задания:**
  - загрузите данные и ознакомьтесь с ними;
  - разделите выборку на обучающую и тестовую;
  - обучите модели k-NN, Decision Tree и SVM. Для k-NN попробуйте найти оптимальное значение k;
  - оцените точность (accuracy) каждой модели на тестовой выборке;
  - сравните результаты и сделайте вывод, какая модель лучше всего справилась с этой задачей.

Код программы:

```
# Лабораторная работа №3. Вариант 1: Iris
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Загрузка и знакомство с данными
```

```
df = pd.read_csv("iris.csv")
print("Размер:", df.shape)
print(df.head(), "\n")
print(df.describe().round(2), "\n")
print("Классы:\n", df["variety"].value_counts(), "\n")
print("Пропуски:", df.isna().sum().sum(), "\n")


X = df.drop(columns="variety")
y = df["variety"]


# 2. Разделение на train/test (70/30, стратификация по классам)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
print("train:", X_train.shape, "test:", X_test.shape, "\n")


# 3. Подбор k для k-NN (кросс-валидация на train + точность на test)
ks = range(1, 31)
cv_acc, test_acc = [], []
for k in ks:
    m = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    cv_acc.append(cross_val_score(m, X_train, y_train, cv=5).mean())
    m.fit(X_train, y_train)
    test_acc.append(accuracy_score(y_test, m.predict(X_test)))


best_k = ks[int(np.argmax(cv_acc))]   # первый k с максимумом CV-точности
print(f"Оптимальное k (по 5-fold CV на train): {best_k}, CV accuracy = {max(cv_acc):.4f}\n")


plt.figure(figsize=(7, 4))
plt.plot(ks, cv_acc, "o-", label="CV (train, 5-fold)")
plt.plot(ks, test_acc, "s--", label="test")
plt.axvline(best_k, color="gray", ls=":", label=f"k = {best_k}")
plt.xlabel("k (число соседей)"); plt.ylabel("Accuracy")
plt.title("k-NN: зависимость точности от k"); plt.grid(alpha=.3); plt.legend()
plt.tight_layout(); plt.savefig("knn_k.png", dpi=150); plt.close()


# Модели
models = {
    f"k-NN (k={best_k})": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=best_k)),
    "Decision Tree": DecisionTreeClassifier(max_depth=3, random_state=42),
    "SVM (RBF)": make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1.0, gamma="scale")),
}


# 4. Оценка на тесте
results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    results[name] = accuracy_score(y_test, pred)
    print("=" * 60)
    print(name, f"-> accuracy = {results[name]:.4f}")
    print("Матрица ошибок:\n", confusion_matrix(y_test, pred))
    print(classification_report(y_test, pred, digits=3))


# Для Decision Tree дополнительно исследуем глубину
print("Decision Tree: влияние max_depth")
for d in [1, 2, 3, 4, 5, None]:
    t = DecisionTreeClassifier(max_depth=d, random_state=42)
    cv = cross_val_score(t, X_train, y_train, cv=5).mean()
    t.fit(X_train, y_train)
    print(f"  max_depth={d}: CV={cv:.4f}, test={accuracy_score(y_test, t.predict(X_test)):.4f}")


# Для SVM — сравнение ядер
print("\nSVM: влияние ядра")
for kern in ["linear", "rbf", "poly"]:
    s = make_pipeline(StandardScaler(), SVC(kernel=kern))
    cv = cross_val_score(s, X_train, y_train, cv=5).mean()
    s.fit(X_train, y_train)
    print(f"  {kern}: CV={cv:.4f}, test={accuracy_score(y_test, s.predict(X_test)):.4f}")


# Устойчивость вывода: 5-fold CV на всей выборке (тест из 45 объектов слишком мал)
print("\n5-fold CV на всей выборке (150 объектов):")
for name, model in models.items():
    sc = cross_val_score(model, X, y, cv=5)
    print(f"  {name}: {sc.mean():.4f} ± {sc.std():.4f}")


# Итоговый график
plt.figure(figsize=(6, 4))
bars = plt.bar(results.keys(), results.values(), color=["#4c72b0", "#55a868", "#c44e52"])
plt.ylim(0.8, 1.02); plt.ylabel("Accuracy (test)"); plt.title("Сравнение моделей")
for b, v in zip(bars, results.values()):
    plt.text(b.get_x() + b.get_width()/2, v + .005, f"{v:.3f}", ha="center")
plt.tight_layout(); plt.savefig("compare.png", dpi=150); plt.close()
```

Результаты работы программы:

1. Данные. Набор Iris содержит 150 объектов, 4 числовых признака (sepal.length, sepal.width, petal.length, petal.width) и целевую переменную variety с тремя классами (Setosa, Versicolor, Virginica) по 50 объектов в каждом. Пропусков нет, классы сбалансированы. Выборка разделена в соотношении 70/30 со стратификацией по классам: 105 объектов в обучающей и 45 в тестовой выборке (random_state = 42).

2. Подбор k для k-NN. Значение k перебиралось от 1 до 30. Для выбора использовалась 5-кратная кросс-валидация на обучающей выборке (признаки стандартизированы). Лучшая CV-точность 0,9714 получена при k = 14, это значение использовано в дальнейшем. На графике показана зависимость точности от k.

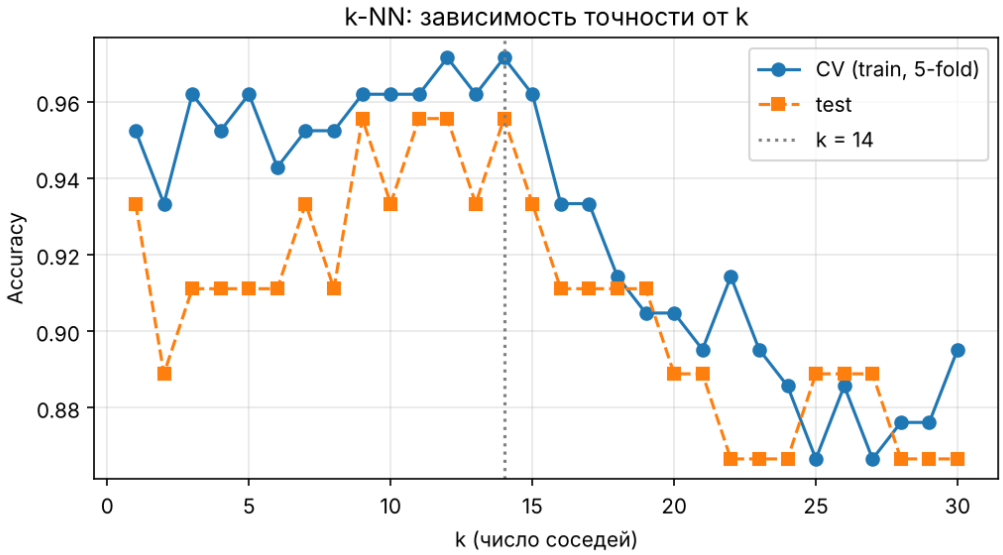


Рис. 1 – Зависимость точности k-NN от числа соседей k (Python)

3. Точность моделей на тестовой выборке:

Модель	Accuracy (тест)	CV на всех 150 объектах
k-NN (k = 14)	0,956	0,940 ± 0,039
Decision Tree (max_depth = 3)	0,978	0,973 ± 0,025
SVM (ядро RBF, C = 1)	0,933	0,967 ± 0,021

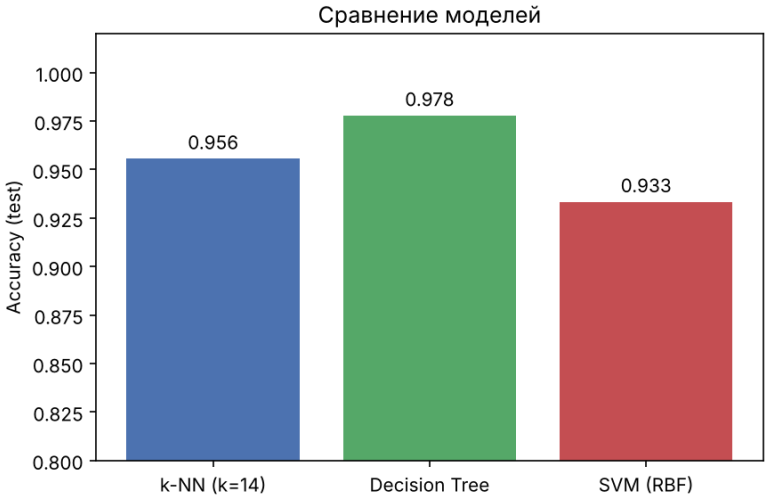


Рис. 2 – Сравнение точности моделей на тестовой выборке (Python)

4. Матрицы ошибок (строки – истинные классы, столбцы – предсказанные; порядок: Setosa, Versicolor, Virginica):

- k-NN: [[15 0 0], [0 15 0], [0 2 13]] – 2 ошибки (два Virginica определены как Versicolor);
- Decision Tree: [[15 0 0], [0 14 1], [0 0 15]] – 1 ошибка (один Versicolor определён как Virginica);
- SVM: [[15 0 0], [0 14 1], [0 2 13]] – 3 ошибки (одна в сторону Virginica, две в сторону Versicolor).

5. Влияние гиперпараметров. Для Decision Tree: при max_depth = 1 CV-точность составляет 0,667 (дерево различает только Setosa), при max_depth = 2 – 0,943, при 3 – 0,952 (максимум), при 4 и больше – 0,943, то есть дальнейшее усложнение дерева не улучшает результат. Для SVM: ядра linear и rbf дают одинаковую CV-точность 0,971, ядро poly хуже – 0,905.

Анализ результатов. Класс Setosa все три модели определяют безошибочно: он линейно отделим от остальных по размерам лепестка. Все ошибки возникают между классами Versicolor и Virginica, значения признаков которых частично перекрываются. На тестовой выборке лучший результат у Decision Tree (0,978), затем k-NN (0,956) и SVM (0,933). Однако тестовая выборка содержит всего 45 объектов, поэтому разница в 1–2 объекта несущественна. По кросс-валидации на всей выборке Decision Tree и SVM практически равны (0,973 и 0,967), k-NN немного уступает и сильнее зависит от выбора k.

Orange:

Схема в Orange построена из виджетов: File (загрузка iris.csv) → Data Table, Scatter Plot, Box Plot (знакомство с данными); File → Data Sampler (70 % обучающая выборка со стратификацией, 30 % тестовая); модели kNN, Tree и SVM → Test and Score (оценка на тестовой выборке, Data Sample → Data, Remaining Data → Test Data) → Confusion Matrix.

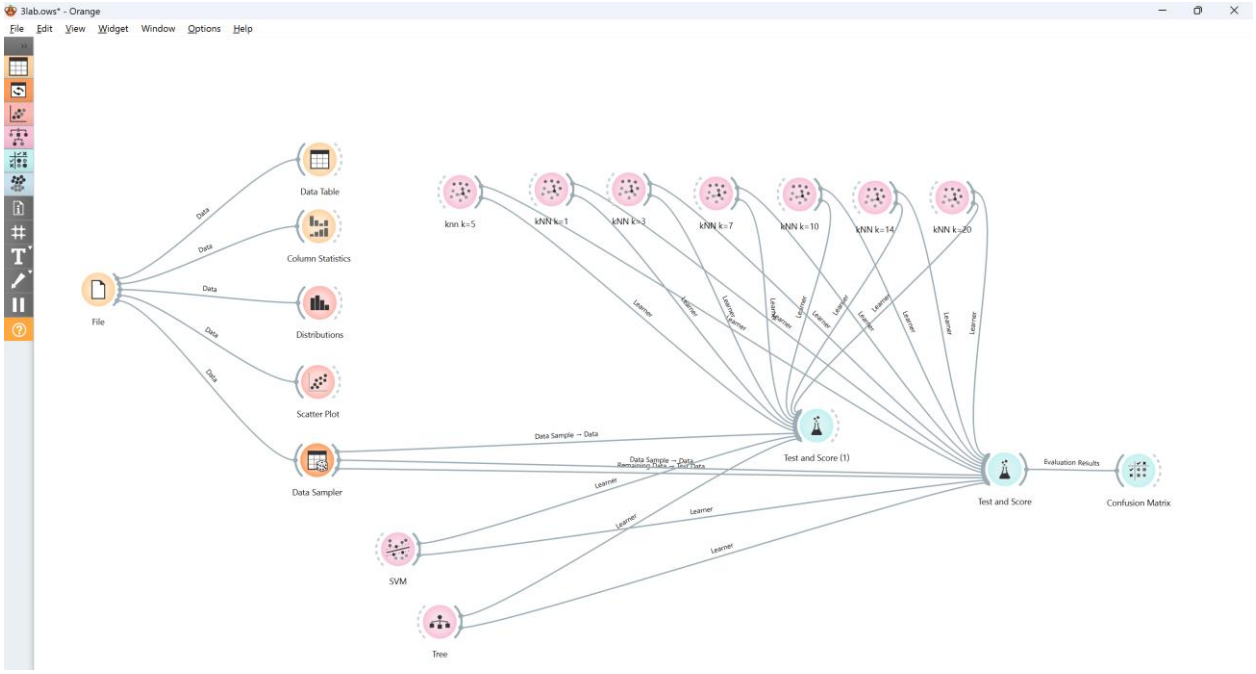
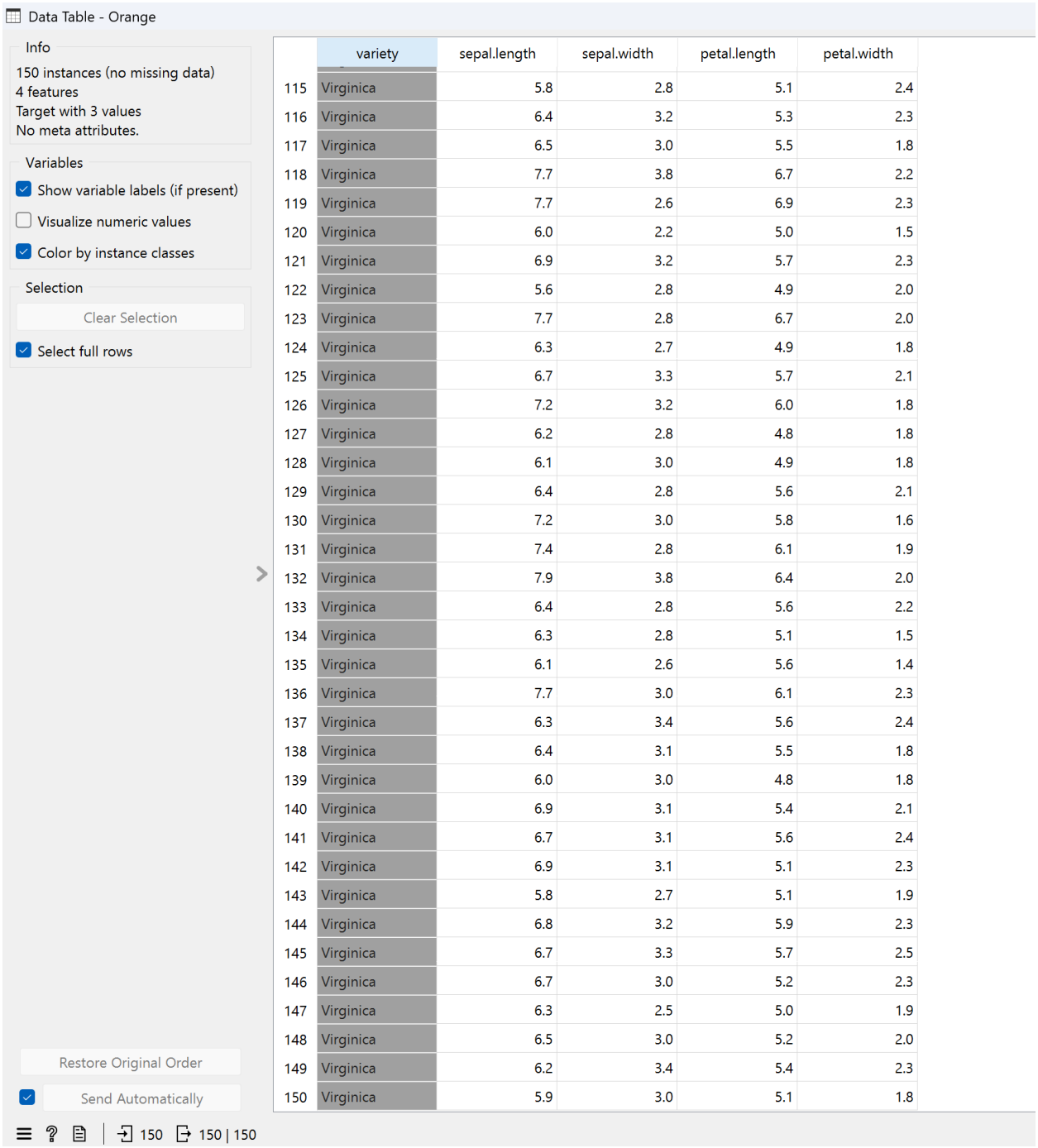


Рис. 3 – Схема лабораторной работы в Orange

Знакомство с данными.



	variety	sepal.length	sepal.width	petal.length	petal.width
115	Virginica	5.8	2.8	5.1	2.4
116	Virginica	6.4	3.2	5.3	2.3
117	Virginica	6.5	3.0	5.5	1.8
118	Virginica	7.7	3.8	6.7	2.2
119	Virginica	7.7	2.6	6.9	2.3
120	Virginica	6.0	2.2	5.0	1.5
121	Virginica	6.9	3.2	5.7	2.3
122	Virginica	5.6	2.8	4.9	2.0
123	Virginica	7.7	2.8	6.7	2.0
124	Virginica	6.3	2.7	4.9	1.8
125	Virginica	6.7	3.3	5.7	2.1
126	Virginica	7.2	3.2	6.0	1.8
127	Virginica	6.2	2.8	4.8	1.8
128	Virginica	6.1	3.0	4.9	1.8
129	Virginica	6.4	2.8	5.6	2.1
130	Virginica	7.2	3.0	5.8	1.6
131	Virginica	7.4	2.8	6.1	1.9
132	Virginica	7.9	3.8	6.4	2.0
133	Virginica	6.4	2.8	5.6	2.2
134	Virginica	6.3	2.8	5.1	1.5
135	Virginica	6.1	2.6	5.6	1.4
136	Virginica	7.7	3.0	6.1	2.3
137	Virginica	6.3	3.4	5.6	2.4
138	Virginica	6.4	3.1	5.5	1.8
139	Virginica	6.0	3.0	4.8	1.8
140	Virginica	6.9	3.1	5.4	2.1
141	Virginica	6.7	3.1	5.6	2.4
142	Virginica	6.9	3.1	5.1	2.3
143	Virginica	5.8	2.7	5.1	1.9
144	Virginica	6.8	3.2	5.9	2.3
145	Virginica	6.7	3.3	5.7	2.5
146	Virginica	6.7	3.0	5.2	2.3
147	Virginica	6.3	2.5	5.0	1.9
148	Virginica	6.5	3.0	5.2	2.0
149	Virginica	6.2	3.4	5.4	2.3
150	Virginica	5.9	3.0	5.1	1.8

Рис. 4 – Таблица данных Iris в Orange

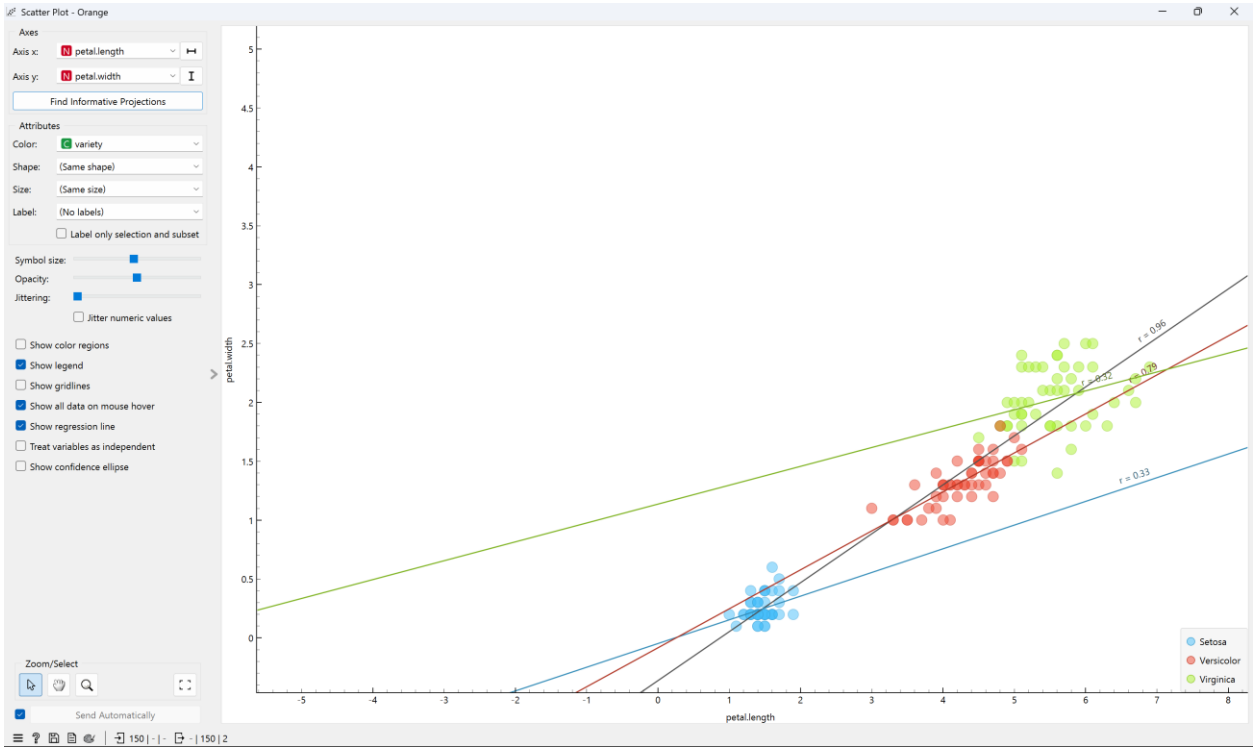


Рис. 5 – Scatter Plot: распределение классов по признакам лепестка

На диаграмме класс Setosa полностью отделён от двух других классов, а Versicolor и Virginica частично перекрываются. Именно в этой области ожидаются ошибки классификации.

Подбор k для k-NN.

Для подбора k на схему добавлено несколько виджетов kNN с разными значениями k (1, 3, 5, 10, 14, 20 и т. д.). Модели сравнивались в Test and Score в режиме кросс-валидации (5 folds) на обучающей выборке.

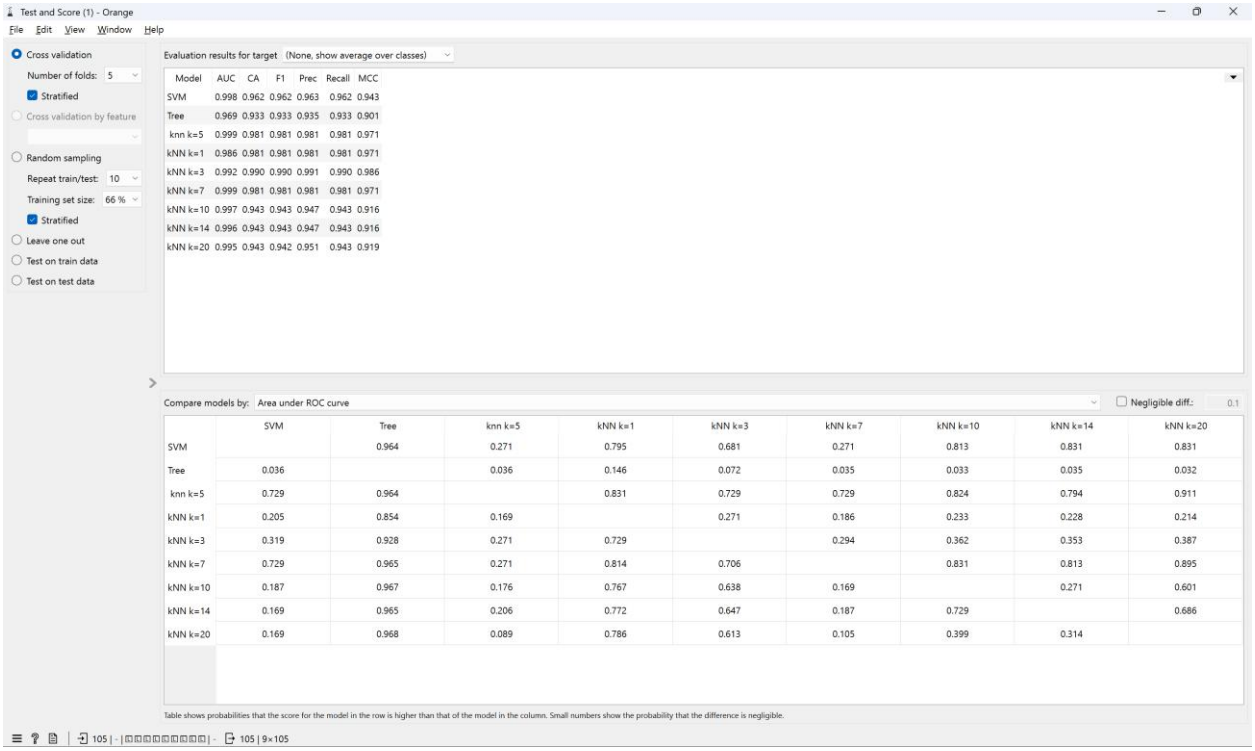


Рис. 7 – Подбор числа соседей k в Orange

Лучшее качество по метрике CA получено при k = **10**.

Оценка моделей на тестовой выборке.

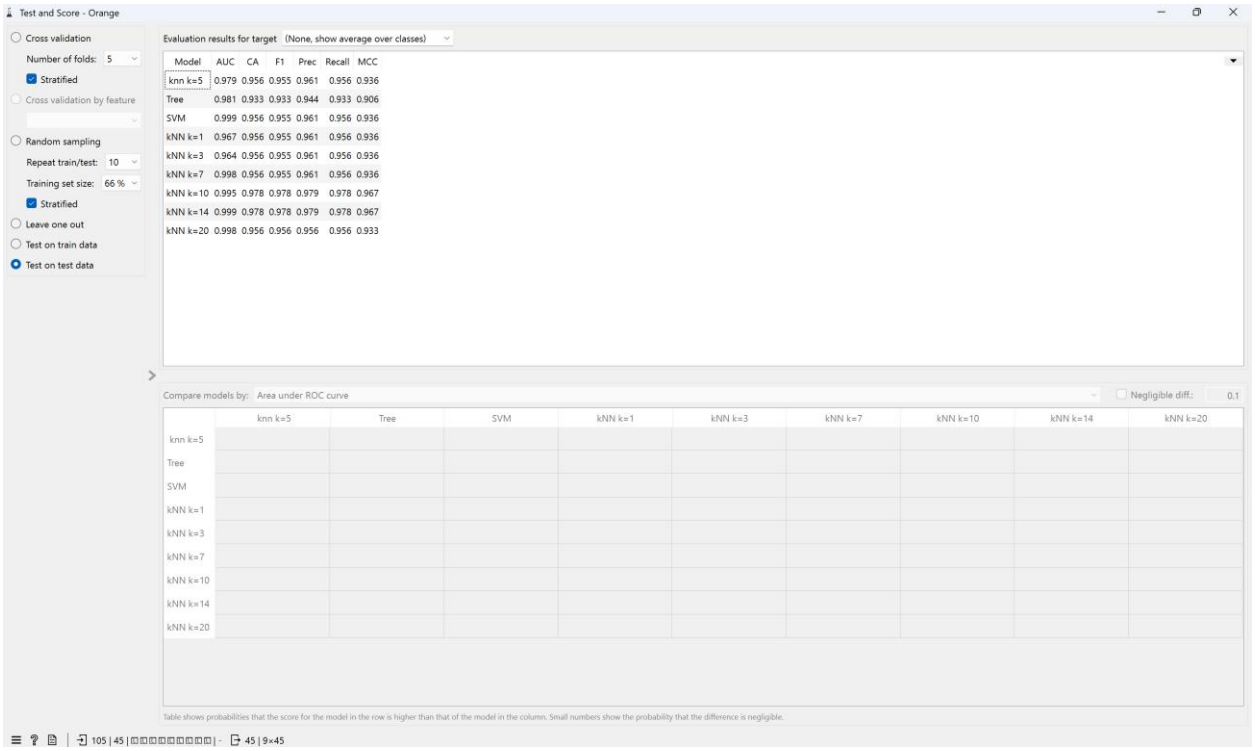


Рис. 8 – Точность моделей на тестовой выборке в Orange

По результатам Test and Score точность (CA) составила: k-NN – **0.956**, Decision Tree – **0.933**, SVM – **0.956**.

Матрицы ошибок.

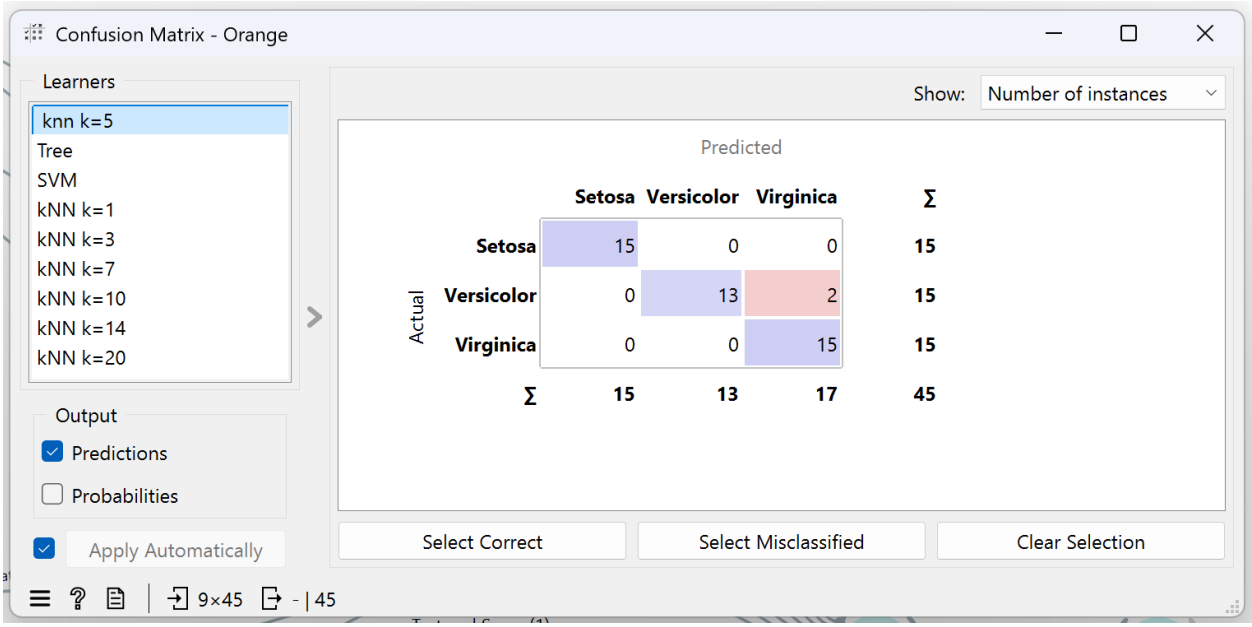


Рис. 9 – Матрица ошибок k-NN в Orange

я

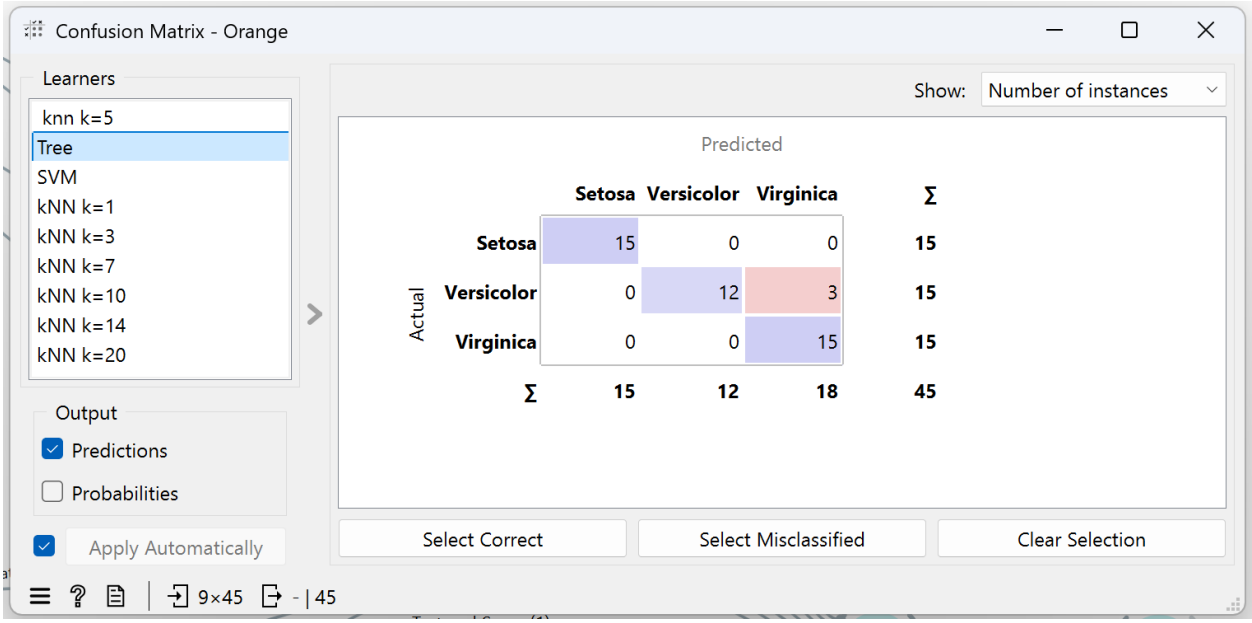


Рис. 10 – Матрица ошибок Decision Tree в Orange

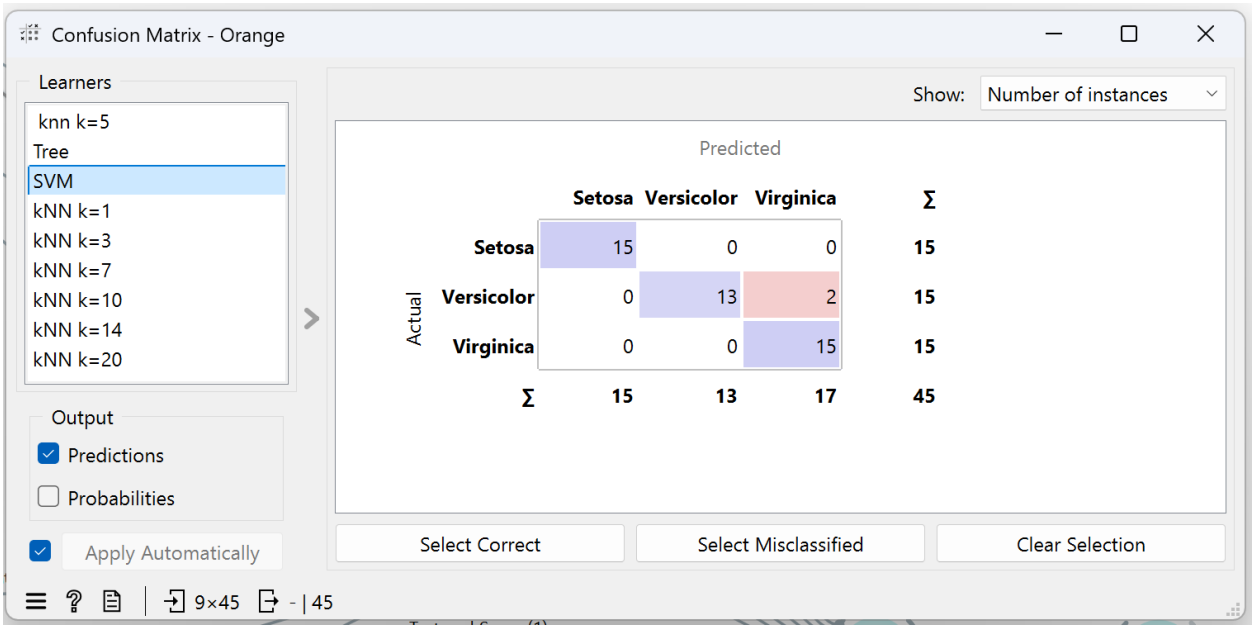


Рис. 11м – Матрица ошибок SVM в Orange

Как и в Python, класс Setosa определяется безошибочно, а ошибки наблюдаются только между Versicolor и Virginica. Значения могут немного отличаться от результатов Python, так как в Orange используется другое разбиение данных и другие значения параметров моделей по умолчанию.

Вывод: на практике сравнил три классических алгоритма классификации – k-NN, Decision Tree и SVM – на наборе данных Iris. Научился подбирать гиперпараметры (число соседей k, глубину дерева, ядро SVM) с помощью кросс-валидации и оценивать их влияние на качество. На тестовой выборке точность составила: k-NN (k = 14) – 0,956, Decision Tree (max_depth = 3) – 0,978, SVM (RBF) – 0,933. Setosa определяется всеми моделями идеально, ошибки возникают только между Versicolor и Virginica. Поскольку тестовая выборка мала (45 объектов), различия между моделями статистически несущественны; по кросс-валидации лучше всего справились Decision Tree (0,973) и SVM (0,967). Для данной задачи применимы все три метода: Decision Tree наиболее интерпретируем, SVM показал наиболее стабильный результат, а k-NN и SVM требуют масштабирования признаков.